`ToListAsync()` command instructs EF Core to generate the SQL query, execute it on the database, and build `RecipeSummaryViewModels` from the data returned.

Listing 12.7 Loading a list of items using EF Core in `RecipeService`

```
public async Task<ICollection<RecipeSummaryViewModel>> GetRecipes()
{
    return await _context.Recipes                                    ❶
        .Where(r => !r.IsDeleted)
        .Select(r => new RecipeSummaryViewModel                      ❷
        {                                                            ❷
            Id = r.RecipeId,                                         ❷
            Name = r.Name,                                           ❷
            TimeToCook = $"{r.TimeToCook.TotalMinutes}mins"          ❷
        })
        .ToListAsync();                                              ❸
}
```

❶ A query starts from a DbSet property.

❷ EF Core queries only the Recipe columns it needs to map the view model correctly.

❸ Executes the SQL query and creates the final view models

Notice that in the `Select` method you convert the `TimeToCook` property from a `TimeSpan` to a `string` by using string interpolation:

```
TimeToCook = $"{r.TimeToCook.TotalMinutes}mins"
```

I said before that EF Core converts the series of LINQ expressions to SQL, but that statement is a half-truth: EF Core can't or doesn't know how to convert some expressions to SQL. *In* those cases, such as this example, EF Core finds the fields from the DB that it needs to run the expression on the client side, selects them from the database, and then runs the expression in C#. This approach lets you combine the power and performance of database-side evaluation without compromising the functionality of C#.